

Ollama Overview: Arsitektur, API, dan Ekosistem untuk Large Language Models Lokal

Sumber: <https://github.com/ollama/ollama>

Tanggal akses: 12 Mei 2026

Pendahuluan

Ollama adalah platform perangkat lunak open-source yang dirancang untuk menjalankan large language models (LLMs) secara lokal pada infrastruktur pengguna. Ollama mengabstraksi kompleksitas inference engine melalui antarmuka baris perintah (CLI) dan REST API yang sederhana, memungkinkan peneliti, pengembang, dan institusi untuk menggunakan kemampuan LLM tanpa ketergantungan pada layanan cloud. Ollama didistribusikan di bawah lisensi MIT dan dikelola sebagai proyek komunitas dengan lebih dari 171.000 bintang di GitHub.

Arsitektur Sistem

Backend Inference: llama.cpp

Ollama dibangun di atas [llama.cpp](#), sebuah inference engine untuk LLMs yang ditulis dalam C++ dengan fokus pada performa tinggi pada hardware konsumen. Keunggulan utama llama.cpp meliputi:

- **Quantized Inference:** Mendukung berbagai format kuantisasi (Q4_0, Q4_K_M, Q5_K_M, Q8_0, dll.) yang mengurangi ukuran model 2-4x dengan degradasi kualitas minimal.
- **Mixed Precision:** Menggunakan floating-point 16-bit (FP16) dan integer 4/8-bit untuk menyeimbangkan akurasi dan kecepatan.
- **Memory Mapping (mmap):** Memuat model langsung dari disk tanpa perlu menyalin seluruhnya ke RAM, memungkinkan model besar berjalan pada sistem dengan resource terbatas.
- **CPU Optimization:** Menggunakan instruksi vektor (AVX2, NEON, SVE) untuk akselerasi pada CPU.
- **Offloading GPU:** Memungkinkan sebagian layer model dijalankan pada GPU untuk mempercepat inference.

Komponen Arsitektur

1. **Ollama Server:** Proses daemon yang berjalan di latar belakang, bertanggung jawab mengelola model dan melayani permintaan API. Mendukung konkurensi dengan memuat beberapa model secara simultan.
2. **Ollama CLI:** Antarmuka baris perintah untuk interaksi langsung dengan server — menjalankan model, membuat model kustom, dan mengelola penyimpanan.
3. **Model Registry:** Ollama secara default terhubung ke registry publik di ollama.com/library, yang menyediakan ribuan model termasuk Llama 3.2, Gemma 3, DeepSeek, Mistral, Qwen, Phi-4, dan banyak lagi.
4. **REST API Layer:** Server HTTP yang menyediakan endpoint untuk chat, generate, embeddings, dan manajemen model.
5. **Modelfile Engine:** Sistem untuk membuat model kustom dengan parameter, template, dan adapter LoRA yang dapat dikonfigurasi.

Dukungan Kuantisasi Model

Kuantisasi mengurangi presisi numerik bobot model untuk menghemat memori dan mempercepat inference. Ollama melalui llama.cpp mendukung beberapa level kuantisasi:

Kuantisasi	Bits per Weight	Ukuran Relatif	Penggunaan Ideal
Q4_0	4.0	~25%	Keseimbangan ukuran dan kualitas terbaik
Q4_K_M	4.5	~28%	Kualitas lebih baik dari Q4_0 dengan ukuran sedikit lebih besar
Q5_K_M	5.0	~35%	Kualitas mendekati FP16 dengan ukuran 65% lebih kecil
Q8_0	8.0	~50%	Kualitas hampir identik dengan FP16
FP16	16.0	100%	Kualitas referensi tertinggi

Endpoint REST API

Server Ollama berjalan pada `http://localhost:11434` dan menyediakan REST API berikut:

Method	Endpoint	Deskripsi
POST	<code>/api/chat</code>	Generate respons dalam format percakapan (chat). Mendukung riwayat pesan, system prompt, dan tools/functions.
POST	<code>/api/generate</code>	Generate respons dari prompt tunggal. Mendukung suffix, images (multimodal), dan format JSON.

POST	/api/embed	Menghasilkan vektor embedding dari teks input. Berguna untuk semantic search dan RAG.
POST	/api/pull	Mengunduh model dari registry. Mendukung download tidak langsung (streaming) dan verifikasi checksum SHA256.
POST	/api/push	Mengunggah model kustom ke registry. Memerlukan autentikasi.
POST	/api/create	Membuat model dari definisi Modelfile. Mendukung pembuatan dari base model, Safetensors, atau file GGUF.
POST	/api/copy	Menyalin model dengan nama baru di penyimpanan lokal.
DELETE	/api/delete	Menghapus model dari penyimpanan lokal.
POST	/api/show	Menampilkan informasi detail model termasuk Modelfile lengkap, parameter, dan template.
GET	/api/tags	Mengembalikan daftar semua model yang tersedia di penyimpanan lokal.
GET	/api/ps	Menampilkan model yang sedang dimuat di memori.
HEAD	/api/blobs/:digest	Memeriksa keberadaan blob model di penyimpanan lokal.

Akselerasi Hardware

Ollama mendukung berbagai backend akselerasi hardware untuk mempercepat inference:

- **NVIDIA CUDA:** Untuk GPU NVIDIA (Tesla, GeForce RTX, Quadro). Diaktifkan secara otomatis jika driver CUDA dan NVIDIA Container Toolkit terdeteksi.
- **AMD ROCm:** Untuk GPU AMD (Radeon RX, Instinct). Memerlukan instalasi ROCm runtime.
- **Apple Metal:** Untuk Apple Silicon (M1, M2, M3, M4) dan GPU AMD di Mac Intel. Diaktifkan secara default pada macOS.
- **Intel Vulkan/SYCL:** Untuk GPU Intel (Arc, Iris Xe). Memerlukan driver Vulkan.

Kinerja dan Optimasi

Manajemen Memori

Ollama menggunakan strategi manajemen memori yang efisien:

- **Model Caching:** Model tetap berada di memori selama `OLLAMA_KEEP_ALIVE` (default 5 menit) setelah request terakhir, mengurangi waktu loading untuk request berulang.
- **Concurrent Model Loading:** Beberapa model dapat dimuat bersamaan dengan batasan yang dikonfigurasi melalui `OLLAMA_MAX_LOADED_MODELS`.

- **Parallel Request Processing:** Mendukung pemrosesan beberapa request secara paralel untuk model yang sama.

Context Window

Parameter `num_ctx` mengontrol ukuran context window (jumlah token yang dapat diproses model dalam satu waktu). Context window yang lebih besar memungkinkan model memahami konteks yang lebih panjang tetapi membutuhkan lebih banyak memori dan waktu komputasi. Hubungan antara ukuran context window dan konsumsi memori bersifat kuadratis pada mekanisme attention.

Keamanan dan Privasi

Ollama dirancang dengan prinsip keamanan dan privasi sebagai prioritas:

- **Inferensi Lokal:** Semua komputasi dilakukan pada perangkat pengguna. Data tidak pernah dikirim ke server eksternal.
- **Tampa Telemetri:** Ollama tidak mengirimkan data penggunaan secara default (dapat diaktifkan secara opsional melalui `OLLAMA_DEBUG`).
- **Isolasi Docker:** Model dan data pengguna diisolasi dalam volume Docker saat menggunakan deployment kontainer.
- **API Lokal:** Server API secara default hanya mendengarkan pada `127.0.0.1` (localhost), tidak dapat diakses dari jaringan eksternal tanpa konfigurasi eksplisit.

Ekosistem Integrasi

Ollama terintegrasi dengan lebih dari 100 tools dan framework dalam ekosistem AI, termasuk:

- **AI Frameworks:** LangChain, LlamaIndex, Haystack, Spring AI, Semantic Kernel, LiteLLM
- **Code Editors:** VS Code (Cline, Continue), JetBrains, Emacs, Obsidian
- **Agent Frameworks:** AutoGPT, crewAI, Claude Code, OpenClaw, OpenCode
- **RAG Systems:** RAGFlow, R2R, MaxKB, ChromaDB, pgvector
- **Monitoring:** OpenLIT, Langfuse, Opik, MLflow Tracing

Perbandingan: Lokal vs Cloud

Aspek	Ollama (Lokal)	API Cloud (OpenAI, dll.)
Privasi Data	Data tetap di perangkat	Data dikirim ke server pihak ketiga
Biaya	Gratis (biaya hardware)	Per-token (komulatif)
Latensi	Bergantung GPU lokal	Bergantung jaringan + server
Ketersediaan	Offline penuh	Memerlukan koneksi internet
Skalabilitas	Terbatas hardware	Hampir tak terbatas
Model Terbaru	Bergantung rilis Ollama	Segera setelah rilis provider
Reproducibility	Seed tetap, deterministik	Bergantung provider (seed mungkin tidak konsisten)

Relevansi untuk Penelitian Tesis

Ollama sangat relevan untuk penelitian di bidang NLP, educational data mining, dan sistem rekomendasi berbasis AI karena:

1. **Reproducibility Penuh:** Pengaturan `seed` dan parameter inference yang deterministik memungkinkan eksperimen direproduksi secara tepat.
2. **Biaya Nol:** Tidak ada biaya API berulang — kritis untuk penelitian dengan ribuan eksperimen.
3. **Privasi Data:** Data penelitian (termasuk data mahasiswa atau institusi) tidak perlu dikirim ke server eksternal, memenuhi persyaratan etika penelitian dan perlindungan data.
4. **Fleksibilitas Model:** Peneliti dapat mengganti model dengan mudah (misal: membandingkan performa Gemma 3 vs Llama 3.2) tanpa perubahan kode signifikan.
5. **Dokumentasi Terbuka:** Sebagai proyek open-source, seluruh kode dan dokumentasi dapat diaudit dan direproduksi.

Dokumen ini dicetak dari sumber online untuk keperluan referensi tesis.

Sumber asli: <https://github.com/ollama/ollama> — Diakses pada 12 Mei 2026.